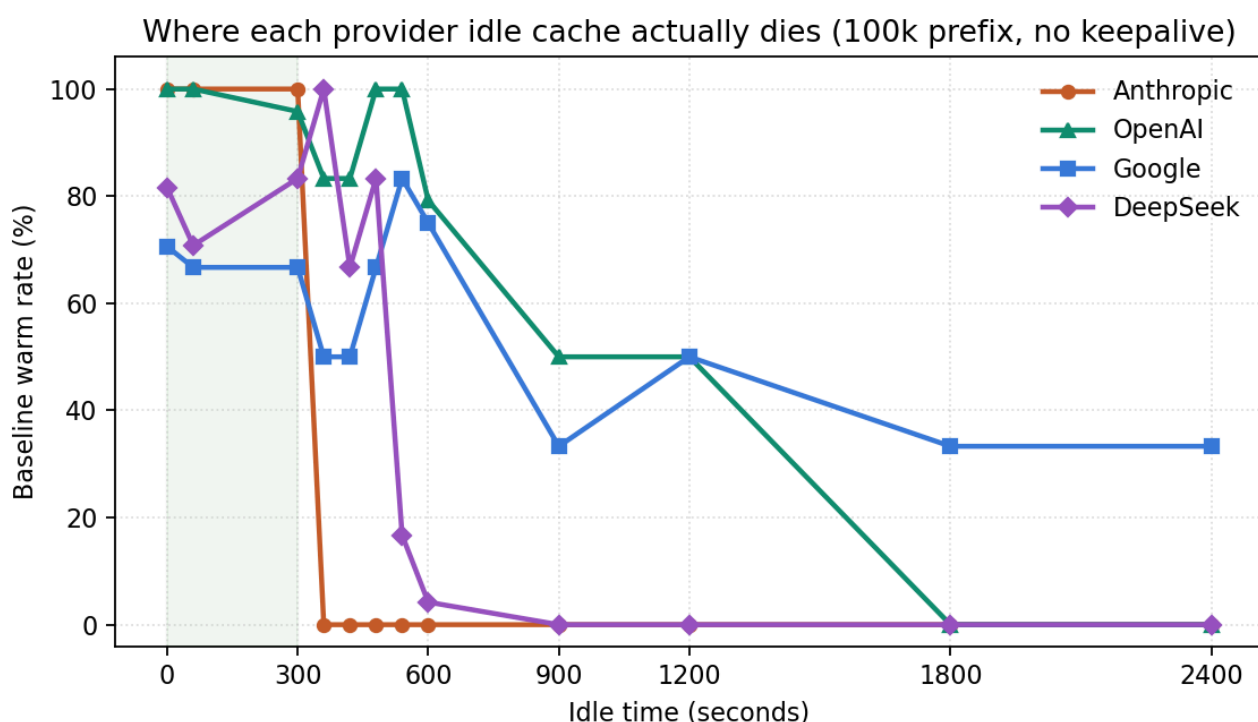


Updated with the full retention curve and the interval frontier: the keepalive saves on Anthropic AND OpenAI inside each provider's paying band, OpenAI's optimal interval is 8 minutes not 4, and an interval past the TTL is actively toxic: every ping re-prefills a dead cache at full price.

这次更新补上了完整的"缓存留存曲线"和"间隔前沿"。结论是：只要落在那家服务商自己的"划算区间"里，Anthropic 和 OpenAI 都能靠保活省钱；OpenAI 的最优间隔是 8 分钟而不是 4 分钟；而一旦间隔超过了 TTL（Time To Live，缓存的存活时限），保活就成了有害操作：每一次 ping，都在按全价把一段已经死掉的缓存重新预填充一遍。



A cache keepalive is the one optimization every agent builder agrees on, and almost everyone runs it at the wrong setting.

缓存保活（cache keepalive，就是隔一段时间给缓存发一次请求，别让它过期）是所有做 agent（智能体，那种能自己调工具、多步骤干活的 AI 程序）的人都认同的优化手段——但几乎所有人都把参数设错了。

The convention is to ping every 30 seconds. That convention costs 8× more than necessary.

大家的习惯做法是每 30 秒 ping 一次。而这个习惯做法花的钱，是必要成本的 8 倍。

I measured keepalive economics across Anthropic, OpenAI, Gemini, and DeepSeek, with a harness that can prove its own timing. The numbers rearranged a few of my beliefs. The right interval is about 4 minutes, not 30 seconds.

我横跨 Anthropic、OpenAI、Gemini 和 DeepSeek 四家，把保活这笔账完整测了一遍。我用了一套测试工具（harness，就是一套固定的测试装置），它能自证每一次调用的时间点。测出来的数字，推翻了我原本的几个想法。正确的间隔大约是 4 分钟，不是 30 秒。

Does the keepalive save you money? That depends on the provider and on how long your agent pauses. The rule: keep the cache warm only when your pause is longer than the provider's eviction point but shorter than its break-even horizon. Inside that window the keepalive pays. On Anthropic and OpenAI it saves real money. On DeepSeek and Gemini it buys only latency. Outside the window it's pure waste.

保活到底能不能帮你省钱？这要看用的是哪家服务商，也要看你的 agent 会停多久。规则是这样的：只有当"停顿时间"比这家的"缓存淘汰点"更长、但又比"回本临界点"更短时，才值得把缓存保热。落在窗口里，保活才划算。在 Anthropic 和 OpenAI 上，它确实能省下真金白银。在 DeepSeek 和 Gemini 上，它买到的只是更低的延迟。一旦出了这个窗口，就是纯浪费。

The pause that eats the cache

那个把缓存吃掉的停顿

Provider prompt caches are one of the best deals in the API: send a prefix the server processed a moment ago, and you pay about a tenth of the input price and skip most of the prefill latency. But the cache expires in minutes, and agentic workloads are the worst possible user of it.

各家服务商的 prompt cache（提示词缓存）是 API 里最划算的交易之一：你发一段服务器刚刚才处理过的前缀（prefix，指对话开头那段固定不变的内容），只要付大约十分之一的输入价格，还能省掉大部分预填充延迟。但问题是，缓存几分钟就过期，而 agent 这类工作负载，恰恰是它最难伺候的用户。

An agent thinks, acts, waits. It fires a request, runs a build or a test suite or sits on a human approval for ten minutes, and only then sends the follow-up that would have reused the (now large) conversation prefix. The pause outlives the cache. The follow-up pays full price and full latency. At agent scale, that's a real line item.

一个 agent 会思考、行动、等待。它发一个请求，然后去跑一次构建、跑一遍测试，或者卡在"等人审批"上十分钟，之后才发下一轮请求——而这一轮本来是可以复用那段（此刻已经很大的）对话前缀的。结果就是：停顿比缓存的寿命还长。这轮后续请求只能付全价、吃全额的延迟。放到 agent 的规模上看，这是一笔实打实的开销。

The defense is well known. Re-send the exact prefix on a timer during the pause; every read refreshes the TTL. Aider shipped it in 2024, Anthropic's docs recommend it, and community posts worked out the Anthropic cost mechanics and even a 4-minute interval on paper. The folklore, it turns out, had part of the right answer. What it didn't have was measurement.

应对办法大家都知道：在停顿期间，用定时器把那段一模一样的前缀重新发一遍；每一次"读缓存"都会把 TTL 刷新一次。Aider（一个开源的 AI 编程助手）在 2024 年就把这个功能做进去了，Anthropic 的官

方文档也推荐这么做，社区里的帖子还算清了 Anthropic 上的成本机制，甚至在纸面上推出了 4 分钟这个间隔。现在看来，这些"民间智慧"里有部分答案是对的。它们缺的，是实测。

Prior to this work there people used arithmetic to guess at the optimal keepalive. Everything following is my measurements with both confirms the optimal time the community found for Anthropic, but also measures the other top provider. I measured four providers, idle gaps out to forty minutes, ping intervals out to fifteen, three independent runs, every call timestamped within the harness.

在这份工作之前，大家是用算术去"猜"最优的保活间隔。而下面所有内容都来自我的实测：它既验证了社区为 Anthropic 找到的最优时间，也把其他几家头部服务商一起测了。我测了四家服务商，空转时间最长测到 40 分钟，ping 间隔最长测到 15 分钟，每组独立跑三遍，而且每一次调用都在测试工具内部打上了时间戳。

I was inspired to do this while in a conference room in Bellevue, at AgentSys, where I saw Haiying Shen and Simon Peter present CacheWise, their study of KV-cache management for coding agents. Their trace analysis shows from the serving side exactly the problem above: agent sessions reuse enormous prefixes, and naive eviction wrecks them. Their question was how the *server* should manage the cache for agents; mine became what the *client* can do about it on its own. The keepalive is the client's answer, and this study is my thanks for the inspiration.

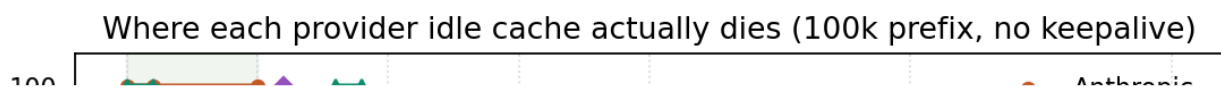
做这件事的灵感，来自我在 Bellevue（贝尔维尤，美国西雅图附近的一座城市）的一间会议室里参加 AgentSys 会议的时候。当时 Haiying Shen 和 Simon Peter 在讲他们的研究 CacheWise，主题是面向编程 agent 的 KV 缓存管理。他们从"服务端视角"做的轨迹分析，正好指出了上面那个问题：agent 会话会复用体量巨大的前缀，而简单粗暴的淘汰策略会把它毁掉。他们关心的是**服务端**该怎么为 agent 管缓存；而我关心的问题变成了：**客户端**自己一个人能做点什么。保活，就是客户端给出的答案。这篇研究，是我对那次启发的一点谢意。

Where each cache actually dies

各家的缓存到底死在什么时候

To know whether a keepalive is worth it, you first have to know when the cache would have died on its own. So I let idle prefixes sit, with no keepalive, and measured the warm rate out to forty minutes. The warm rate is the probability the provider will keep your cache warm during the idle time.

想知道保活划不划算，你得先知道：缓存自己会在什么时候死。所以我让空闲的前缀就那么放着，不做任何保活，然后一直测到 40 分钟，看"命中率"怎么变。这里说的命中率，指的是在你空转的这段时间里，服务商把你的缓存保住的概率。



Idle cache survival with no keepalive, 100k prefix. The green band is the window where every provider is still warm.

不做保活时，空闲缓存的存活情况，前缀长度 10 万 token。绿色那一段表示：在这个窗口里，所有服务商的缓存都还热着。

Two things fall out of this curve, and both matter.

这条曲线能得出两个结论，两个都很重要。

First: the documented TTL is exactly what you can rely on, and no more. Nothing evicted before its documented TTL, and nothing past it is guaranteed. Anthropic's cache falls off a cliff between 5 and 6 minutes, exactly at its advertised 5-minute TTL. No grace period. DeepSeek is gone by 10. OpenAI actually outlives its documented 5 to 10 minutes, still half warm at 20, before going fully cold by 30. A pleasant bonus, but not one you can plan around. Google never converges at all, hovering at a 33 to 83 percent hit rate at every gap. That's not a retention curve. That's a routing lottery.

第一：官方文档写的 TTL，就是你能指望的全部，别指望更多。 没有任何一家会在文档标注的 TTL 之前就把缓存清掉，但超过 TTL 之后，也没有任何保证。Anthropic 的缓存在 5 到 6 分钟之间直接断崖式下跌，正好卡在它官方标的 5 分钟 TTL 上——没有宽限期。DeepSeek 到 10 分钟就没了。OpenAI 倒是活得比它文档写的 5 到 10 分钟更久，到 20 分钟还有一半是热的，直到 30 分钟才彻底冷掉。这是个意外之喜，但你不能拿它做规划。Google 则完全不存在“收敛”这回事，在任何时间间隔下命中率都在 33% 到 83% 之间飘——那不是一条留存曲线，那是在抽签。

So the retention window you can assume by default is the shortest documented TTL in your fleet, minus a margin. Warmth below it held everywhere I measured. Warmth above it is luck.

所以，默认情况下你能假定的留存窗口，就是你所用的服务商里最短的那个官方 TTL，再减掉一点余量。在我测过的所有服务商身上，低于这条线的缓存都还热着；高于这条线，热不热全看运气。

Second, and this corrected my own earlier conclusion: OpenAI is not sticky, it is slow. When I first measured only to ten minutes, OpenAI looked like it never evicted. It does. I just had not waited long enough. That distinction turns out to decide whether the keepalive ever pays for it.

第二点，也纠正了我自己早先的结论：OpenAI 不是"不会淘汰"，而是"淘汰得慢"。 我一开始只测到 10 分钟，那时候 OpenAI 看起来像从不淘汰缓存。它是会的，只是我没等够时间。而正是这个区别，决定了保活对 OpenAI 到底能不能回本。

The keepalive itself works everywhere, and that part was never in doubt. Wherever a baseline drops, the keepalive holds the same prefix warm:

保活本身在哪儿都有效，这一点从来没人怀疑。凡是基线命中率往下掉的地方，保活都能把同一段前缀保热：

Baseline (dashed) vs keepalive (solid) warm rate, out to 30 minutes at 100k. Anthropic: 0 of 48 warm at ten minutes without the keepalive, 40 of 40 with it. Every provider that evicts is held at 100%.

基线（虚线）与保活（实线）的命中率对比，前缀 10 万 token，一直测到 30 分钟。Anthropic：不做保活时，10 分钟那一档 48 个样本里 0 个还热；做了保活，40 个样本 40 个都热。凡是会淘汰缓存的服务商，保活后都能稳在 100%。

The baseline here means there was no keepalive to keep the cache warm so the provider will evict the cache after some time. The real question isn't whether the keepalive works. It's whether it's worth paying for. And that depends on where you sit on the retention curve above.

这里的"基线"指的是：没有保活去维持缓存热度，所以过一段时间服务商就会把缓存淘汰掉。真正的问题从来不是保活有没有用，而是它值不值得你花钱。而这，取决于你落在上面那条留存曲线的什么位置上。

The interval is the whole game

间隔就是全部的关键

Here is the entire economics of the keepalive, and it is settled arithmetic:

保活的经济账，说到底就是下面这几条，而且都是板上钉钉的算术：

1. A ping costs the read price ($\approx 10\%$ of input) every interval τ . Holding a prefix costs $0.1 \times$ per τ .

2. 每 ping 一次，就要付一次"读缓存"的价格（大约是输入价的 10%），每个间隔 τ 付一次。也就是说，维持一段前缀的热度，每过一个 τ 就花掉 0.1 倍输入价。
3. A re-prefill costs 100% once (125% on Anthropic, which bills the cache write). The re-prefill is when the cache has been evicted and the provider needs to do the prefill again.
4. 一次"重新预填充"要付 100% 的钱（在 Anthropic 上是 125%，因为缓存写入也要计费）。所谓重新预填充，就是缓存已经被淘汰了，服务商不得不把这活儿从头再干一遍。
5. So spend per hour falls as $1/\tau$, and the interval buys nothing except safety. The optimal interval is the largest one safely under the provider's retention: ≈ 4 minutes at Anthropic's 5-minute TTL. (Hold that thought; the last section shows what "safely under" is worth, and what happens when you cross it.)
6. 所以每小时的花费按 $1/\tau$ 下降，间隔除了"保险"之外没买到任何别的东西。最优间隔，就是"安全地小于服务商留存时间"的那个最大间隔：在 Anthropic 5 分钟的 TTL 下，大约就是 4 分钟。（先记住这一点；最后一节会讲"安全地小于"到底值多少钱，以及一旦越线会发生什么。）
7. Break-even is idle $\approx \tau(w/r - 1)$: about 46 minutes at Anthropic's prices, 36 for OpenAI and DeepSeek, 12 for Google (its cached reads are $0.25\times$, not $0.1\times$). Past that, stop pinging: let the cache die and pay the re-prefill.
8. 回本点是空闲时间 $\approx \tau(w/r - 1)$ ：按 Anthropic 的价格算大约 46 分钟，OpenAI 和 DeepSeek 是 36 分钟，Google 是 12 分钟（它读缓存是 0.25 倍价，不是 0.1 倍）。超过这个点，就别 ping 了：让缓存自然死掉，回来时付那次重新预填充的钱更划算。

That last one is the rule worth understanding. Think of every ping as a cost you pay to avoid one the re-prefill. On Anthropic the cost is $0.1\times$ the input price and the re-prefill is $1.25\times$, so the re-prefill is worth about twelve pings. Pay one ping every 4 minutes and the re-prefill exceed the cost after roughly eleven of them: **46 minutes** of pause. That's the line you don't want to cross.

最后这条规则最值得弄明白。你可以把每一次 ping 看成"为了躲开一次重新预填充"而付的钱。在 Anthropic 上，一次 ping 是 0.1 倍输入价，一次重新预填充是 1.25 倍，也就是说，一次重新预填充大约等于十二次 ping 的钱。你每 4 分钟付一次 ping，等 ping 的钱超过重新预填充的钱，大约需要十一次，也就是 **46 分钟**的停顿。这条线，就是你不该越过的。

Concretely, on a 100k-token Anthropic prefix (about \$0.30 of input at list price): each ping costs \$0.03, and a cold re-prefill costs \$0.38.

具体来说，拿一段 10 万 token 的 Anthropic 前缀举例（按标价算，输入大约 0.3 美元）：每次 ping 花 0.03 美元，而一次冷启动的重新预填充要花 0.38 美元。

- **10-minute pause**: two pings, \$0.06, and you skip the \$0.38. Keep it warm. You are $6\times$ ahead.
- **停顿 10 分钟**：ping 两次，花 0.06 美元，省掉了那 0.38 美元。保热它，你净赚 6 倍。
- **46-minute pause**: eleven pings, \$0.33, versus the \$0.38 re-prefill. Roughly a wash. This is the line.
- **停顿 46 分钟**：ping 十一次，花 0.33 美元，对上一次 0.38 美元的重新预填充。差不多打平。这就是那条临界线。

- **1-hour pause:** fifteen pings, \$0.45, to avoid a \$0.38 re-prefill. Stop pinging. Let the cache go cold and pay the re-prefill when you come back.
- **停顿 1 小时:** 为了躲开 0.38 美元的重新预填充, 要 ping 十五次、花 0.45 美元。停了吧。让缓存冷掉, 等你回来时直接付那次重新预填充的钱。

The formula tells you where your own line sits once you plug in your prices and interval. The point is that *there is* a line. It's closer than people assume. And past it, the disciplined move is to stop paying premiums on a policy you will never claim.

把你自己的价格和间隔代进公式, 它就会告诉你你的线在哪儿。关键就在于: 这条线是存在的。而且它比大家以为的要近得多。越过它之后, 最克制的做法就是——别再为一个你根本不会去理赔的保险付保费了。

The 30-second convention everyone copies has a break-even of about 6 minutes, which is why it loses money at a 10-minute gap on all four providers. I ran the 4-minute arm too: pings fired at +240.1s and +480.1s (0.1s drift), held 23 of 24 samples warm, and cost **7.8× less** than the 30-second arm for the same warmth. The convention isn't cautious. It's just expensive.

那个人人照抄的 30 秒习惯, 回本点只有大约 6 分钟, 所以它在四家服务商上、面对 10 分钟的停顿全都是亏钱的。我也跑了 4 分钟那一组: ping 分别在 +240.1 秒和 +480.1 秒触发 (偏差 0.1 秒), 24 个样本里 23 个保持热, 而且达到同样的热度, 花的钱比 30 秒那一组 **少了 7.8 倍**。这个习惯做法不是"保守"。它只是单纯很贵。

600s idle, 100k prefix	Let it die	Keepalive 30s	Keepalive 240s
Anthropic Sonnet 4.5	\$0.667 (cold)	\$0.867 (loses money)	\$0.414 (saves 38%)
OpenAI GPT-5.1	\$0.115 (usually warm anyway)	\$0.317 (loses)	\$0.136 (loses)
DeepSeek V3.2	\$0.043 (cold, 5.4s TTFT)	\$0.249 (loses)	\$0.060 (loses; but ~3× faster)
Gemini 2.5 Pro	\$0.131 (usually warm anyway)	\$0.649 (loses)	\$0.186 (loses)

空闲 600 秒, 前缀 10 万 token	任它冷掉	30 秒保活	240 秒保活
Anthropic Sonnet 4.5	\$0.667 (冷)	\$0.867 (亏钱)	\$0.414 (省 38%)
OpenAI GPT-5.1	\$0.115 (反正通常还热)	\$0.317 (亏)	\$0.136 (亏)
DeepSeek V3.2	\$0.043 (冷, TTFT 即首字延迟 5.4 秒)	\$0.249 (亏)	\$0.060 (亏; 但快约 3 倍)
Gemini 2.5 Pro	\$0.131 (反正通常还热)	\$0.649 (亏)	\$0.186 (亏)

Look at the last column. At a 10-minute pause, only Anthropic and DeepSeek have evicted, so only they have anything to insure. Even there, the 30-second convention loses money.

看最后一列。在 10 分钟停顿的情况下，只有 Anthropic 和 DeepSeek 真的把缓存淘汰了，所以也只有它们有东西可"投保"。可即便在这两家，30 秒的习惯做法照样是亏钱的。

That is what I first published: at 10 minutes, only Anthropic actually saved. But that was a fact about 10 minutes, not about the providers. OpenAI and Gemini were still warm at 10 minutes, so of course keeping them warm bought nothing. The real question: what happens once they evict?

这就是我最初发表的内容：在 10 分钟这个点上，只有 Anthropic 真的省到了钱。但那是"10 分钟"这个条件的事实，不是这几家服务商的事实。OpenAI 和 Gemini 在 10 分钟时缓存还热着，那保活自然买不到任何东西。真正的问题是：等它们真的开始淘汰缓存之后，会怎样？

The paying band, measured

实测出来的"划算区间"

So I ran it again at a 30-minute pause, where the retention curve says every provider except Gemini is fully cold. Now there is a re-prefill to avoid on all of them. Same 4-minute keepalive, same 100k prefix: 于是我把停顿拉到 30 分钟，又跑了一遍——按留存曲线，这时候除了 Gemini，其他几家的缓存都已经彻底冷掉了。这下它们全都有一次重新预填充等着躲。还是同样的 4 分钟保活，还是同样的 10 万 token 前缀：

1800s idle, 100k prefix	Let it die	Keepalive 240s	Verdict
Anthropic Sonnet 4.5	\$0.333 (cold, 2.9s TTFT)	\$0.214	saves 1.56x
OpenAI GPT-5.1	\$0.104 (cold, 4.4s TTFT)	\$0.085	saves 1.23x
DeepSeek V3.2	\$0.022 (cold, 5.4s TTFT)	\$0.108	loses; buys latency

空闲 1800 秒，前缀 10 万 token	任它冷掉	240 秒保活	结论
Anthropic Sonnet 4.5	\$0.333 (冷，首字延迟 2.9 秒)	\$0.214	省 1.56 倍
OpenAI GPT-5.1	\$0.104 (冷，首字延迟 4.4 秒)	\$0.085	省 1.23 倍
DeepSeek V3.2	\$0.022 (冷，首字延迟 5.4 秒)	\$0.108	亏；买到的是延迟

There it is. **Once OpenAI actually evicts, its keepalive saves money too.** The keepalive was never an Anthropic-only trick. It pays for any provider once your pause is long enough to reach eviction and short enough to stay under break-even.

答案出来了。一旦 OpenAI 真的开始淘汰缓存，它的保活也一样能省钱。保活从来就不是只有 Anthropic 才玩得转的把戏。只要你的停顿"长到能触发淘汰、又短到没超过回本点"，它对任何一家服务

商都划算。

Each provider has a band where this holds: Anthropic from about 5 to 46 minutes; OpenAI from about 30 minutes out to over an hour at its own best interval (next section). Outside the band, on either side, you are paying premiums for nothing.

每家服务商都有这么一段"成立区间": Anthropic 大约是 5 到 46 分钟; OpenAI 从大约 30 分钟起, 一直到一小时以上 (用它自己的最佳间隔算, 下一节会讲)。出了这个区间, 无论是偏短还是偏长, 你都是在白付保费。

DeepSeek is a real exception. It does evict, but a cold re-prefill costs two cents, so the pings cost more than the eviction they prevent no matter how you time them. Its keepalive is a latency play only: 5.4 seconds to first token down to 2.0.

DeepSeek 是真正的例外。它确实会淘汰缓存, 但一次冷的重新预填充只要两美分, 所以不管你怎么安排时间, ping 的钱都比它挡下的那次淘汰更贵。对 DeepSeek 来说, 保活只是一笔"买速度"的买卖: 首次出词时间从 5.4 秒降到 2.0 秒。

Gemini is worse: it never reliably evicts (its hit rate is a routing lottery, not a retention curve) and its cached reads cost $0.25\times$ instead of $0.1\times$, so there is no pause length where keeping it warm pays on cost. For those two, keep warm only if you are buying speed, never dollars.

Gemini 更糟: 它从不稳定地淘汰缓存 (它的命中率是路由抽签, 不是留存曲线), 而且读缓存要 0.25 倍价、不是 0.1 倍, 所以根本不存在哪个停顿长度能让保热在成本上划算。对这两家来说, 只有当你是在买速度、而不是在省钱的时候, 才值得保热。

Past the TTL, pings turn toxic

一旦越过 TTL, ping 就变成毒药

One question remained. If keepalive cost falls as the interval grows, why stop at 4 minutes? So I kept going: same 30-minute pause, same prefix, ping intervals of 4, 8, and 15 minutes.

还剩一个问题。如果保活的成本随着间隔变长而下降, 那为什么停在 4 分钟? 于是我接着往下测: 同样的 30 分钟停顿, 同样的前缀, ping 间隔分别取 4 分钟、8 分钟和 15 分钟。

Marginal cost of the pause at each ping interval, with warm counts. Dashed line: just letting the cache die.

不同 ping 间隔下, 这段停顿的边际成本, 以及对应的命中数量。虚线表示: 直接让缓存死掉的情况。

The interval is bounded by the provider's own retention, and the chart shows both sides of that line.

间隔的上限，是被服务商自己的留存时间卡住的，而这张图把这条线的两边都展示出来了。

OpenAI's cache holds 8-minute gaps, so 8-minute pings halve its cost again: \$0.042 against a \$0.104 re-prefill, a $2.45\times$ saving, double what 4-minute pings deliver. Its true optimal interval is about 8 minutes, not 4.

OpenAI 的缓存能扛住 8 分钟的空档，所以 8 分钟 ping 一次，成本又砍掉一半：0.042 美元对上一次 0.104 美元的重新预填充，省了 2.45 倍，是 4 分钟 ping 的两倍。它真正的最优间隔大约是 8 分钟，不是 4 分钟。

Anthropic at 8-minute pings is a catastrophe, and the reason matters. Its cache dies at 5 minutes, so every 8-minute ping lands on a dead cache and re-prefills it at full write price. Three pings, three full prefills, and you still arrive warm only by accident of timing. The bill: \$1.334, four times the cost of never pinging at all. An interval past the TTL isn't merely wasteful. Every ping pays the exact penalty it was meant to avoid.

而 Anthropic 用 8 分钟 ping，就是一场灾难，原因很关键。它的缓存在 5 分钟就死了，所以每 8 分钟一次的 ping 全都打在了了一具尸体上，然后按全价重新预填充一遍。三次 ping，三次全价预填充，最后你能赶上热缓存纯粹是碰运气。账单是 1.334 美元，是完全不 ping 的四倍。间隔一旦超过 TTL，就不只是"浪费"——每一次 ping，都在支付它本来想躲开的那笔罚金。

Nobody survives 15-minute pings, and DeepSeek at 8 minutes is a coin flip (its cache dies around 9). So the sharpened rule: **the economical interval is your provider's own measured retention minus a margin, and it is per-provider.** Four minutes on Anthropic. Eight on OpenAI. A multi-provider agent that uses one global ping interval is either overpaying on OpenAI or burning money on Anthropic.

15 分钟 ping 谁也扛不住，而 DeepSeek 用 8 分钟是掷硬币（它的缓存大约 9 分钟死）。所以规则可以打磨得更锋利：**最经济的间隔，等于你家服务商实测的留存时间减去一点余量，而且必须按服务商分别设置。** Anthropic 上是 4 分钟，OpenAI 上是 8 分钟。一个跨多家服务商跑的 agent，如果只用一套全局 ping 间隔，那它要么在 OpenAI 上多花了钱，要么在 Anthropic 上烧了钱。

What is settled, and what I believe

哪些是确定的事实，哪些是我的判断

Settled: the arithmetic above, the measured retention curves, the measured $7.8\times$, the break-even horizons, and the interval frontier with its toxic edge. Those you can check against the data, which is public with the harness.

确定的事实包括：上面那些算术、实测出的留存曲线、实测到的 7.8 倍、各个回本点，以及那道带着"中毒边缘"的间隔前沿。这些都可以拿数据去核对，数据和测试工具都是公开的。

Belief: what happens when everyone takes this advice. A cache tier's eviction policy ranks by expected reuse; a keepalive manufactures recency, so once every client pings, LRU has nothing left to rank and the tier degrades for everyone. Residency today is priced per read, not per token-hour held, so each operator's rational rent payment imposes an unpriced cost on every other tenant. My prediction: keepalive adoption forces providers to meter residency directly. Google's explicit cache already bills per token-hour, and Anthropic's 1-hour tier at $2\times$ write is a step in the same direction. The arbitrage is real, and it has an expiry date.

属于我个人判断的是：当所有人都照这个建议做之后，会发生什么。缓存层的淘汰策略是按“预期会被复用的概率”来排序的；而保活人为地制造出“刚被用过”的假象，所以一旦每个客户端都在 ping，LRU（Least Recently Used，最近最少使用，一种优先淘汰最久没被用过的内容的策略）就再没什么可排的了，整个缓存层对所有租户都退化了。现在的计费方式是“按读取次数收费”，而不是“按你占用了多少 token·小时收费”，所以每个使用者在理性地付自己那份“租金”时，都会把一笔没有定价的成本转嫁给其他所有租户。我的预测是：保活的普及会逼着服务商直接按“缓存驻留时长”来计费。Google 的显式缓存已经在按 token·小时计费了，Anthropic 那个写入收 2 倍价的 1 小时档，也是朝同一个方向迈的一步。这个套利机会是真实存在的，但它有保质期。

The policy

行动准则

- **Keep warm only across bounded, plausibly reused pauses** (a tool call, an approval wait), at τ^* set per provider to its measured retention minus a margin. About 4 minutes on Anthropic, 8 on OpenAI, 50 on Anthropic's 1-hour tier.
- **只在有明确边界、且大概率会被复用的停顿期间保热**（比如一次工具调用、一次等待审批），间隔 τ^* 按服务商分别设为“实测留存时间减去余量”。Anthropic 上大约 4 分钟，OpenAI 上 8 分钟，Anthropic 的 1 小时档是 50 分钟。
- **Never let τ exceed the provider's retention.** Past it, every ping re-prefills a dead cache at full price and the keepalive costs more than doing nothing ($4\times$ more, on Anthropic at 8-minute pings).
- **绝不要让 τ 超过服务商的留存时间。** 一旦超过，每次 ping 都在按全价重新预填充一段死缓存，保活比什么都不干还贵（在 Anthropic 上用 8 分钟 ping，是贵 4 倍）。
- **Abandon the keepalive past the break-even horizon** (≈ 46 minutes at Anthropic prices). Let it die, re-prefill, walk away.
- **超过回本临界点就放弃保活**（按 Anthropic 的价格大约是 46 分钟）。让它死掉，重新预填充一次，然后走人。
- **Stay inside the paying band.** Keep warm only when your pause is longer than the provider's eviction point (Anthropic ~ 5 min, OpenAI ~ 30 min, DeepSeek ~ 10 min, Gemini never) and shorter

than its break-even. Anthropic and OpenAI save money inside their bands. DeepSeek and Gemini never save on cost, only latency.

- 待在"划算区间"里。只有当停顿比服务商的淘汰点更长（Anthropic 约 5 分钟，OpenAI 约 30 分钟，DeepSeek 约 10 分钟，Gemini 则压根没有）、又比回本点更短时，才保热。Anthropic 和 OpenAI 在各自区间里是省钱的。DeepSeek 和 Gemini 从来省不了成本，只能省延迟。
- **Below the eviction point, do not bother.** The cache is still warm on its own, so the pings buy nothing. This is the mistake the 30-second convention makes on every provider.
- 在淘汰点以下，别费这个劲。缓存自己还热着呢，ping 什么都买不到。这正是 30 秒习惯做法在每一家服务商身上都会犯的错。
- **Never keep a dead session warm.** The pi agent harness implements exactly this: pings only while a tool batch runs, default off.
- 绝不给一个已经死掉的会话保活。pi agent harness 就是这么实现的：只有在一批工具调用执行期间才 ping，默认关闭。

Keep it warm, briefly, at your provider's own rate. Ping less, but never slower than the cache forgets.

短时间地保热，按你家服务商自己的节奏来。少 ping 一点，但永远别慢过缓存遗忘的速度。

References

参考资料

- Shubham Tiwari, Tapan Chugh, Nash Rickert, Simon Peter, Ratul Mahajan, Haiying Shen. CacheWise: Understanding Workloads and Optimizing KVCache Management for Efficiently Serving LLM Coding Agents (the talk that inspired this project).
- Shubham Tiwari, Tapan Chugh, Nash Rickert, Simon Peter, Ratul Mahajan, Haiying Shen. CacheWise: 理解工作负载并优化 KV 缓存管理，以高效服务 LLM 编程 agent（启发了本项目的那场演讲）。
- Maxim Khailo. Keeping the Cache Warm Pays: Keepalive Economics for Agentic Workloads (arXiv:2607.19214) and the harness + data (scripts/cache-research).
- Maxim Khailo. 保热缓存是划算的：面向 agent 工作负载的保活经济学 (arXiv:2607.19214)，以及测试工具 + 数据 (scripts/cache-research)。
- Aider AI. Change history (cache keepalive, v0.53.0, 2024).
- Aider AI. 变更历史（缓存保活，v0.53.0，2024 年）。
- Anthropic. Prompt caching (pre-warming guidance).
- Anthropic. 提示词缓存（预热指导）。
- Veritas Supera. We Taught Our AI Agents to Take Coffee Breaks (the 4-minute interval, on paper).

- Veritas Supera。我们教会了 AI agent 去喝咖啡休息（在纸面上推出 4 分钟间隔的那篇）。
- Brandon Wie. Anthropic Prompt Cache TTL + Cost Mechanics.
- Brandon Wie。Anthropic 提示词缓存的 TTL 与成本机制。
- OpenClaw. Cache keep-warm pings proposal (TTL-proportional intervals).
- OpenClaw。缓存保热 ping 提案（按 TTL 比例设置间隔）。
- Don't Break the Cache (what to cache for agentic tasks, a different question).
- 别打破缓存（agent 任务该缓存什么，这是另一个问题）。
- Prompt cache thrashing (the commons externality).
- 提示词缓存抖动（公地外部性问题）。

Your Agentic Workflow's Cache Keepalive Costs 8x Too Much (v2: the interval frontier)

Maxim Khailo · 24 Jul 2026 · 10 min read · 2026-07-24

<https://blog.mempko.com/your-agentic-workflows-cache-keepalive-costs-8x-too-much-v2-the-interval-frontier/>

